1

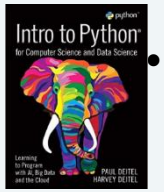
Programmeren voor fysici

1.Inleiding Linux

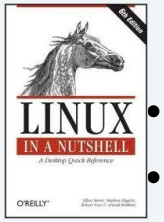
© 2008 Pearson Education, Inc. All rights reserved.

Lesmateriaal en bijkomende informatie

- Basis Boeken:



- Intro to Python, 6/e Harvey M. Deitel and Paul J. Deitel, Pearson (2020)



- Linux in a nutshell Ellen Siever, O'Reilly 1999
- Gratis op: http://docstore.mik.ua/orelly/linux/lnut/

Internet:

https://docs.python.org/3/

https://docs.python.org/3/reference/index.html

https://docs.python.org/3/library/index.html

Python interpreter en programmeeromgeving:

Deze cursus is gebaseerd op Python versie 3.6 of hoger

https://visualstudio.microsoft.com/vs/community/

https://www.python.org/downloads/

© 2008 Pearson Education, Inc. All rights reserved.

Doelstelling

- Voorkennis
  - Basis syntax van PYTHON
  - Basisconcepten van object georienteerd programmeren
- Leerresultaten:
  - Werken met het Linux operating system
  - Opbouwen van programmeervaardigheid
  - Begrip en gebruik van gevorderde concepten, software design etc.
  - Gebruik van bestaande softwarebibliotheken
  - Bij het oplossen van fysische problemen

© 2008 Pearson Education, Inc. All rights reserved.

Voorkennis

- PYTHON syntax en basisconcepten van OOP
  - Fundamentele data types
  - Variable scope
  - Functies (declaraties en implementaties) en functie (operator) overloading
  - Controle structuren (selectie en loops)
  - Klassen: Encapsulatie en data hiding
  - Inheritance
  - PYHTON libraries en data structuren

© 2008 Pearson Education, Inc. All rights reserved.

Doelstelling

- Werkwijze:
  - 20 u hoorcollege (Linux, scripting, herhaling basisconcepten PYTHON, gevorderde concepten, overzicht van externe bibliotheken, …)
  - 20 u oefensessies (Tussentijdse opdrachten die leiden tot het eindproject, …)
  - 20 u collectief eindproject met tussentijdse bijeenkomsten voor discussie, evaluatie, …

© 2008 Pearson Education, Inc. All rights reserved.

Evaluatie

- GEEN mondeling of schriftelijk eindexamen
- Evaluatie is WEL INDIVIDUEEL
- Tussentijdse oefeningen zijn verplicht en worden individueel ingediend en gekwoteerd voor een vastgelegde deadline.
- Quotering oefeningen: Samen bedragen zij 50% van je eindresultaat. Overleg met medestudenten wordt aangemoedigd.

© 2008 Pearson Education, Inc. All rights reserved.

Eindwerk

- Collectieve eindopdracht wordt gedurende de cursus besproken. Eindproduct dient klaar te zijn na de Paasvakantie en staat eveneens op 50% van de punten.
- Evaluatie: In teams van 2-3 personen, op basis van eindproduct
- Eindproduct:
  - Alle broncode en gecompileerde bibliotheken nodig voor de presentatie van het eindresultaat.
  - Een collectief verslag waarin de probleemstelling, de oplossingsstrategie en de resultaten worden besproken.
  - Een presentatie met demo van de software

© 2008 Pearson Education, Inc. All rights reserved.

Verdere afspraken

- Lesmateriaal, opdrachten en mededelingen worden verspreid via Blackboard
- Opdrachten worden ook ingeleverd via Blackboard
- Practica zijn gereserveerd om aan opdrachten en eindproject te werken, niet om te chatten, emails lezen, web surfen, etc...
- Vragen stellen kan tijdens de les, de practica, of na afspraak

© 2008 Pearson Education, Inc. All rights reserved.

Inhoud cursus

- Werken met Linux
  - Linux commando's
  - Shells en shell scripts
  - Compilers en makefiles
- Gevorderde PYTHON concepten en goed programmeren
  - Robuust ontwerp en ontwerppatronen
  - Dieper inzicht in de Python library
  - Gebruik van externe bibliotheken
  - Exception handling
  - Multithreading vs multiprocessing
  - …

© 2008 Pearson Education, Inc. All rights reserved.

Inhoud

1. Inleiding
   1.1 Linux operating system
   1.2 Linux file system
   1.3 Linux command syntax & Shells
   1.4 Shell scripting
2. Programmeren onder Linux
   2.1 File editing met sed en awk
   2.2 De PYTHON Interpreter
   2.3 PYTHON libraries en IDE
3. Robuust ontwerp in PYTHON
4. Data science met PYTHON
5. Machine Learning met SciKit, Keras en TensorFlow
6. Andere topics: Concurrency en HPC

© 2008 Pearson Education, Inc. All rights reserved.

1.1 Linux

Voor meer info zie: http://tldp.org/LDP/intro-linux/html/

- 1969: Bell labs ontwikkelt UNIX als hardware onafhankelijk operating systeem voor mainframe computers
- Jaren 1980: IBM introduceert eerste PC's: opkomst van MS-DOS en Macintosh
- 1991: Linus Torvalds (Helsinki University) ontwikkelt een UNIX OS voor PC's: Linux
  - Vooral eerst populair in academics, gebruikt als netwerk server, weinig 'desktop' applicaties
  - Sinds +- 15j enorm in populariteit gegroeid: Linux heeft de grootste hardware coverage

© 2008 Pearson Education, Inc. All rights reserved.

1.1 Waarom Linux

- Pro:
  - Open source: gratis + miljoenen gebruikers kunnen mee ontwikkelen
  - Maakt optimaal gebruik van hardware: heeft weinig geheugen en CPU nodig
  - Is in staat op op allerlei soorten hardware te werken, is zeer schaalbaar
  - Stabiel en veilig: weinig systeemonderbrekingen, zeer uitgebreide veiligheidscontrole
  - Multi-user en multi-tasking
  - Linux is het OS op 91% van de Top 500 supercomputers
- Contra:
  - Tekst gebaseerde interactie met systeem vraagt iets steilere leercurve
  - Vroeger heel wat problemen met randapparatuur: videokaarten, USB, camera's
  - Er zijn heel veel Linux distributies, elk met hun eigen accenten/applicaties:
    - 3 families: Debian (Ubuntu), Slackware (SUSE), RedHat (Fedora)
    - Verschillen zitten in de applicaties en ondersteuning van randapparatuur
- Linux operating system structuur:
  - Kernel: de basis van het OS, afhankelijk van de processor architectuur (bvb x86 instruction set architecture (Intel, AMD, 32 vs 64 bit))
  - Basisprogramma's: Veelal onder de GNU licence: bvb: http://packages.debian.org/stable/

© 2008 Pearson Education, Inc. All rights reserved.

1.1 Interactie met Linux

- Elke gebruiker logt in met gebruikersnaam en paswoord
- Via een terminal window (xterm)
- Volledige controle van het systeem mogelijk, mits root privileges
  - root = speciale gebruiker: superuser of administrator
- Interactie met het Linux systeem gebeurt via een command interpreter: de shell
  - Bourne Again shell (bash)
  - C shell (csh)
  - Enhanced C shell (tcsh)
- Shells dienen voor:
  - interactief gebruik
  - aanpassingen van je Linux sessie
  - Programmeren (shell scripting)
- Basis commando's omvatten het manipuleren van bestanden en folders en opvragen van informatie/documentatie

© 2008 Pearson Education, Inc. All rights reserved.

1.2 Linux filesystem

- In Linux is nagenoeg alles een file: commando's zijn uitvoerbare files die in speciale folders staan: zie volgende slide
- Speciale folders: / = root folder (begin van de boomstructuur), . = huidige folder, .. = bovenliggende folder
- Controle over files en folders is erg uitgebreid in Linux:
  - 3 niveaus van toegang: gebruiker, groep, anderen
  - 3 soorten toegang: read, write, execute
  - 7 types files:
  - d = directory
  - - = regular file
  - l = symbolic link
  - s = Unix domain socket
  - p = named pipe
  - c = character device file
  - b = block device file

- Verkrijg informatie door ls commando
- Verander permissie met chmod

Commando: vb chmod u+x <file>

R = read permissie

W = write permissie

X = execute permissie

User permissies Group permissies Others permissies

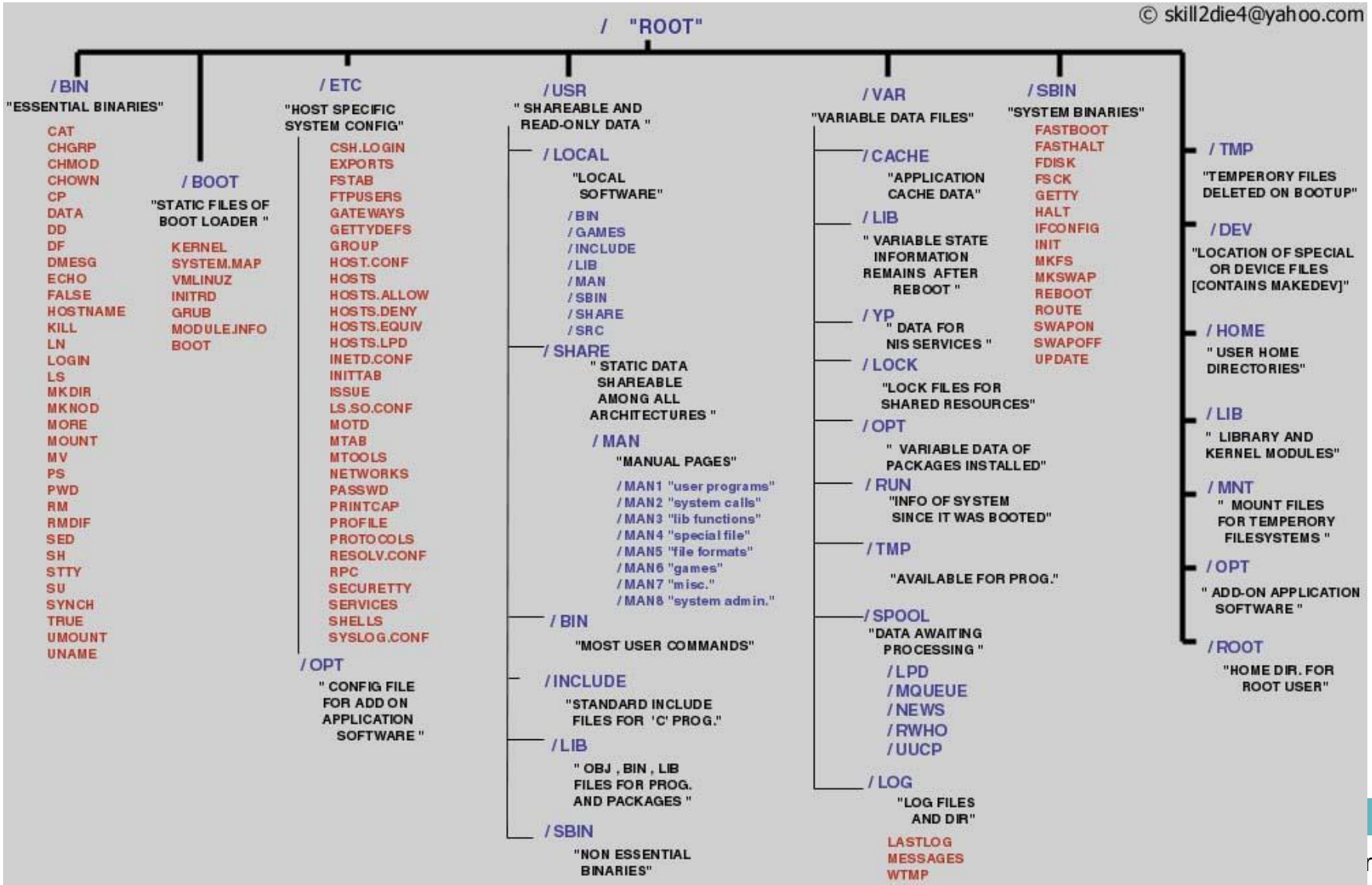
file type

```
remortel@remortel:~$ ls -l
total 88
-rwxr-xr-x  1 remortel remortel 13396 2011-01-19 19:33 cisco-decrypt
-rw-r--r--  1 remortel remortel  3282 2011-01-19 19:32 cisco-decrypt.c
drwxr-xr-x  4 remortel remortel  4096 2011-01-20 20:39 Desktop
drwxr-xr-x  2 remortel remortel  4096 2010-10-20 12:36 Documents
drwxr-xr-x  2 remortel remortel  4096 2011-01-24 11:46 Downloads
-rw-r--r--  1 remortel remortel   179 2010-10-20 12:30 examples.desktop
drwxr-xr-x 11 remortel remortel  4096 2010-12-04 12:42 g4work
-rw-r--r--  1 remortel remortel    28 2010-10-25 19:08 geantinstall.txt
-rw-r--r--  1 remortel remortel   210 2010-11-17 16:18 higz_windows.dat
drwxr-xr-x 43 remortel remortel  4096 2011-01-21 12:59 ISMD
-rw-r--r--  1 remortel remortel   127 2010-11-17 16:20 last.kumac
drwxr-xr-x  2 remortel remortel  4096 2011-01-12 14:30 lic
drwxr-xr-x  2 remortel remortel  4096 2010-10-20 12:36 Music
-rw-r--r--  1 remortel remortel     0 2010-11-17 16:18 paw.metafile
-rwxr-xr-x  1 remortel remortel  3845 2009-09-01 01:58 pcf2vpnc
drwxr-xr-x  2 remortel remortel  4096 2010-10-20 12:36 Pictures
drwxr-xr-x  2 remortel remortel  4096 2010-10-20 12:36 Public
drwxr-xr-x  2 remortel remortel  4096 2010-10-20 12:36 Templates
drwxr-xr-x  2 remortel remortel  4096 2010-10-20 12:36 Videos
-rw-r--r--  1 remortel remortel   672 2011-01-19 19:22 VPNUA-NEW.pcf
```

2008 Pearson Education, Inc. All rights reserved.

1.2 Linux filesystem

- Organisatie van standaard folders onder Linux



rved.

1.2 Zoeken naar files

- Vinden van files en commando's
  - pwd: waar bevind ik mij in de file structuur
  - ls : lijst bestanden in deze folder
    - Speciale folders:
      - ~ : je home folder
      - ./ : de folder waar je jezelf bevindt
      - ../: de folder boven de locatie waar je je bevindt
      - / : de root folder (hoofd van de file hierarchie)
  - cd <path/foldernaam>: begeef je naar de desbetreffende folder
  - type: doorzoekt alle uitvoerbare files, inclusief standaard commando's, aliases en folders in jouw $PATH en geeft informatie over locatie en aanpassingen
  - apropos: wordt gebruikt indien je de precieze naam van een programma niet kent, maar ongeveer weet wat het doet
  - apropos doorzoekt alle man pages naar de zoekterm die jij opgeeft

© 2008 Pearson Education, Inc. All rights reserved.

1.2 Zoeken naar files

```
CLEAR: command not found
remortel@remortel:~$ clear
remortel@remortel:~$ pwd
/home/remortel
remortel@remortel:~$ ls .
cisco-decrypt    examples.desktop  last.kumac    Pictures
cisco-decrypt.c  g4work            lic           Public
Desktop          geantinstall.txt  Music         Templates
Documents        higz_windows.dat  paw.metafile  Videos
Downloads        ISMD              pcf2vpnc      VPNUA-NEW.pcf
remortel@remortel:~$ ls ../
freevo  remortel
remortel@remortel:~$ type ls
ls is aliased to `ls --color=auto'
remortel@remortel:~$ type -a ls
ls is aliased to `ls --color=auto'
ls is /bin/ls
remortel@remortel:~$ apropos browser
aa-update-browser (8) - update browser profiles with browser abstractions
assistant (1)         - on-line documentation browser for Qt4
assistant-qt4 (1)     - on-line documentation browser for Qt4
charmap (1)           - Unicode character picker and font browser
gnome-character-map (1) - Unicode character picker and font browser
google-chrome (1)     - the web browser from Google
gucharmap (1)         - Unicode character picker and font browser
infobrowser (1)       - read Info documents
libsmbclient (7)      - An extension library for browsers and that can be used as a generic browsing API.
links (1)             - lynx-like alternative character mode WWW browser
sensible-browser (1)  - sensible editing, paging, and web browsing
smbtree (1)           - A text based smb network browser
viewres (1)           - graphical class browser for Xt
w3m (1)               - a text based Web browser and pager
www-browser (1)       - lynx-like alternative character mode WWW browser
remortel@remortel:~$ type -a google-chrome
google-chrome is /usr/bin/google-chrome
remortel@remortel:~$
```

© 2008 Pearson Education, Inc. All rights reserved.

1.2 Zoeken op inhoud megt grep

- Syntax: *grep [options] PATTERN [file ...]*
  - Voorbeelden van [options]:
    -i (negeer lowercase/uppercase verschil);
    -v (invert selectie: zoek alles waar niet PATTERN in voorkomt
  - PATTERN: kan string zijn of regular expression (zie later)
  - [file …]: kan een bestandsnaam zijn, of lijst van bestandsnamen gescheiden door spaties, of een directory of lijst van directories zijn

Meer info via *man grep*

© 2008 Pearson Education, Inc. All rights reserved.

1.2 Man pages en $PATH

- Elk programma heeft een man page, behalve sommige ingebouwde commando's:
  - Bevat een beschrijving van het programma en illustreert het gebruik en opties
  - Gebruik als man <commando>
- $PATH: een lijst van locaties waar de shell automatisch in zoekt naar commando's, bibliotheken, manpages beschrijvingen etc.
  - Een shell variabele (waarde van variabele PATH wordt opgevraagd via $PATH)

Toewijzing van een variabele via '=' operator
Opvragen waarde van variabele via $naam
echo: een eenvoudig commando om tekst naar het scherm te printen

```
remortel@remortel:~$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games
remortel@remortel:~$ type VTKData.readme
bash: type: VTKData.readme: not found
remortel@remortel:~$ PATH=$PATH:/opt/vtk561
remortel@remortel:~$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/opt/vtk561
remortel@remortel:~$ type VTKData.readme
VTKData.readme is /opt/vtk561/VTKData.readme
remortel@remortel:~$
```

© 2008 Pearson Education, Inc. All rights reserved.

1.3 Linux Shells

- Shell = de interface tussen gebruiker en operating system
- Doel: interactief gebruik, aanpassen van je Linux omgeving, programmeren (scripting)
- Shell flavors:
  - Bash: standaard shell met
    - input/output faciliteiten
    - Metakarakters (wildcards) voor afkorting van bestandsnamen
    - Shell variabelen
    - Programmeermogelijkheden via controle structuren
    - Algebra met gehele getallen
    - Afkortingen voor commando's via aliasen
    - Job control
    - …
  - C shell (csh)
  - Enhanced C shell (tcsh)
  - Korn shell (ksh)
- Indien bash niet je standaard shell is, kies voor bash via

Commando: exec bash

© 2008 Pearson Education, Inc. All rights reserved.

1.3 Linux command syntax

- Basiscommando's: onafhankelijk van de shell

Command	Description	DOS equivalent
ls	lists the content of a directory	dir
cd	change directory	cd
cd ..	parent directory	cd..
mkdir	creates a new directory	md
rmdir	eliminates a directory	deltree
cp	copy a file	copy, xcopy
mv	moves a file	move
rm	removes a file	del
passwd	changes the user's password	
cat	displays the file's content	type
more	displays the file's content with pauses	type |more
man **apropos**	help for the requested command	help
lpr	prints the requested file	print

© 2008 Pearson Education, Inc. All rights reserved.

1.3 Linux command syntax

- Basiscommando's: onafhankelijk van de shell

Command	Meaning
>	Redirect output
>>	Append to file
<	Redirect input
<<	"Here" document (redirect input)
|	Pipe output
&	Run process in background.
;	Separate commands on same line
*	Match any character(s) in filename
?	Match single character in filename
[ ]	Match any characters enclosed
( )	Execute in subshell
' '	Substitute output of enclosed command
" "	Partial quote (allows variable and command expansion)
' '	Full quote (no expansion)
\	Quote following character
$var	Use value for variable
$$	Process id
$0	Command name
$n	nth argument (n from 0 to 9)
$*	All arguments as a simple word

Command	Meaning
#	Begin comment
bg	Background execution
break	Break from loop statements
cd	Change directories
continue	Resume a program loop
echo	Display output
eval	Evaluate arguments
exec	Execute a new shell
fg	Foreground execution
jobs	Show active jobs
kill	Terminate running jobs
newgrp	Change to a new group
shift	Shift positional parameters
stop	Suspend a background job
suspend	Suspend a foreground job
time	Time a command
umask	Set or list file permissions
unset	Erase variable or function definitions
wait	Wait for a background job to finish

© 2008 Pearson Education, Inc. All rights reserved.

1.3 Linux command syntax

- Basiscommando's: onafhankelijk van de shell

Command	Meaning
>	Redirect output
>>	Append to file
<	Redirect input
<<	"Here" document (redirect input)
|	Pipe output
&	Run process in background.
;	Separate commands on same line
*	Match any character(s) in filename
?	Match single character in filename
[ ]	Match any characters enclosed
( )	Execute in subshell
` `	Substitute output of enclosed command
" "	Partial quote (allows variable and command expansion)
' '	Full quote (no expansion)
\	Quote following character
$var	Use value for variable
$$	Process id
$0	Command name
$n	nth argument (n from 0 to 9)
$*	All arguments as a simple word

Command	Meaning
#	Begin comment
bg	Background execution
break	Break from loop statements
cd	Change directories
continue	Resume a program loop
echo	Display output
eval	Evaluate arguments
exec	Execute a new shell
fg	Foreground execution
jobs	Show active jobs
kill	Terminate running jobs
newgrp	Change to a new group
shift	Shift positional parameters
stop	Suspend a background job
suspend	Suspend a foreground job
time	Time a command
umask	Set or list file permissions
unset	Erase variable or function definitions
wait	Wait for a background job to finish

- Voor overzicht zie:

http://www.oreillynet.com/linux/cmd/

http://www-zeuthen.desy.de/dv/documentation/unixguide/

© 2008 Pearson Education, Inc. All rights reserved.

1.3 Linux command syntax

- Keyboard shortcuts

Key or key combination	Function
Ctrl+A	Move cursor to the beginning of the command line.
Ctrl+C	End a running program and return the prompt, see Chapter 4.
Ctrl+D	Log out of the current shell session, equal to typing exit or logout.
Ctrl+E	Move cursor to the end of the command line.
Ctrl+H	Generate backspace character.

© 2008 Pearson Education, Inc. All rights reserved.

1.3 Redirect

- Standaard schrijft de shell de output van een commando naar het scherm (stdout) en leest ze input van het toetsenbord
- Output omleiden naar een file via > <file>
- Files worden standaard overschreven, gebruik >> om de file aan te vullen
- Indien je zowel output als error logs wil wegschrijven naar dezelfde file gebruik dan &> of &>>
- Speciale file /dev/null : indien je wil dat de output nergens naartoe gaat

```
remortel@remortel:~/bin$ ls > ~/output.txt
remortel@remortel:~/bin$ cd ..
remortel@remortel:~$ ls >> ~/output.txt
remortel@remortel:~$ cat output.txt
interactive
interactive~
printtest
printtest~
bin
cisco-decrypt
cisco-decrypt.c
Desktop
Documents
Downloads
examples.desktop
```

© 2008 Pearson Education, Inc. All rights reserved.

1.3 Pipes

- Wanner je de output van een commando direct wil doorgeven als input aan een ander commando
- Bvb: ls ~/. > inhoud.txt; sort inhoud.txt

is equivalent aan

ls ~/. | sort

```
remortel@remortel:~/test$ ls -a
.  ..  .a  ..foo  .normal_dot_file  normal_file
remortel@remortel:~/test$ ls -a > out.txt
remortel@remortel:~/test$ sort out.txt
.
..
.a
..foo
.normal_dot_file
normal_file
out.txt
remortel@remortel:~/test$ ls -a | sort
.
..
.a
..foo
.normal_dot_file
normal_file
out.txt
remortel@remortel:~/test$
```

© 2008 Pearson Education, Inc. All rights reserved.

1.3 Subshell

- Je wil een pipe maken met een commando dat enkel file namen als argunt neemt
  - Bvb: find . -name 'normal*' | rm (werkt dit??)
  - Oplossing: subshell ( )
    - rm $(find . -name 'normal*')

In dit geval bewaar je de output van de subshell in een variabele

© 2008 Pearson Education, Inc. All rights reserved.

1.3 Aanhalingstekens

- Probleem: Hoe maak je het onderscheid tussen reguliere tekst en speciale karakters of linux commando's ?
  - Bvb Hoe print je 'A coffee costs 5$?!' op het scherm?
- Enkele aanhalingstekens voor letterlijke strings
- Indien men letterlijke strings met shell expansies wil mengen gebruikt men dubble aanhalingstekens voor de expansie, of een \ voor het letterlijk over te nemen speciale karakter

```
remortel@remortel:~$ echo A coffee is $5?!
A coffee is ?!
remortel@remortel:~$ echo 'A coffee is $5?!'
A coffee is $5?!
remortel@remortel:~$ echo 'A coffee is $5 for $USER ?!'
A coffee is $5 for $USER ?!
remortel@remortel:~$ echo 'A coffee is $5 for '$USER' ?!'
A coffee is $5 for remortel ?!
remortel@remortel:~$ echo A coffee is \$5 for $USER \?\!
A coffee is $5 for remortel ?!
remortel@remortel:~$ echo '$USER won't pay $5 for coffee.'
> ^C
remortel@remortel:~$ echo "$USER won't pay $5 for coffee."
remortel won't pay  for coffee.
remortel@remortel:~$ echo "$USER won't pay \$5 for coffee."
remortel won't pay $5 for coffee.
remortel@remortel:~$ echo 'I won'\''t pay $5 for coffee.'
I won't pay $5 for coffee.
remortel@remortel:~$
```

© 2008 Pearson Education, Inc. All rights reserved.

1.3 Filename xpansion & wildcards

*: eender welk en willekeurig hoeveel (ook nul) karakters

?: eender welk enkelvoudig karakter

[*lijst*]: eender welk karakter uit de *lijst*

[b-d]: eender welk karakter tussen b en d

[!*k*]: alles behalve karakter k

```
remortel@remortel:~/test$ ls -a
.  ..  .a  ..foo  .normal_dot_file  normal_file
remortel@remortel:~/test$ ls -d .*
.  ..  .a  ..foo  .normal_dot_file
remortel@remortel:~/test$ ls -d .[!.]*
.a  .normal_dot_file
remortel@remortel:~/test$ ls -d .??*
..foo  .normal_dot_file
remortel@remortel:~/test$ ls -d .[!.]* .??*
.a  ..foo  .normal_dot_file  .normal_dot_file
remortel@remortel:~/test$ ls -d .[!.]* .??* | sort
.a
..foo
.normal_dot_file
.normal_dot_file
remortel@remortel:~/test$
```

© 2008 Pearson Education, Inc. All rights reserved.

1.3 Uitvoeren en verificatie van programma's

Probleem: ga na of een programma met succes werd uitgevoerd, en doe een handeling die afhangt van dit succes

Oplossing: de shell variabele $? is een integer waarde tussen 0 en 255 (o= succes, 1-255 is error code)

```
remortel@remortel:~$ ls bin
interactive  interactive~  printtest  printtest~  readdtest  readdtest~
remortel@remortel:~$ echo $?
0
remortel@remortel:~$ ls onbestaand
ls: cannot access onbestaand: No such file or directory
remortel@remortel:~$ echo $?
2
remortel@remortel:~$ ls bin ; if (( $? )); then echo "failed"; else echo "succes"; fi
interactive  interactive~  printtest  printtest~  readdtest  readdtest~
succes
remortel@remortel:~$ ls onbestaand ; if (( $? )); then echo "failed"; else echo "succes"; fi
ls: cannot access onbestaand: No such file or directory
failed
remortel@remortel:~$
```

© 2008 Pearson Education, Inc. All rights reserved.

1.3 Sequentieel uitvoeren van programma's

Probleem: je wil prog2 uitvoeren wanner prog1 met succes is voltooid, of je wil een actie ondernemen wanneer prog1 niet met succes is voltooid

Oplossing: gebruik && voor succes, en || voor falen

```
remortel@remortel:~$ ls bin
interactive  interactive~  printtest  printtest~  readdtest  readdtest~
remortel@remortel:~$ echo $?
0
remortel@remortel:~$ ls onbestaand
ls: cannot access onbestaand: No such file or directory
remortel@remortel:~$ echo $?
2
remortel@remortel:~$ ls bin ; if (( $? )); then echo "failed"; else echo "succes"; fi
interactive  interactive~  printtest  printtest~  readdtest  readdtest~
succes
remortel@remortel:~$ ls onbestaand ; if (( $? )); then echo "failed"; else echo "succes"; fi
ls: cannot access onbestaand: No such file or directory
failed
remortel@remortel:~$ ls onbestaand && echo "success"
ls: cannot access onbestaand: No such file or directory
remortel@remortel:~$ ls bin && echo "success"
interactive  interactive~  printtest  printtest~  readdtest  readdtest~
success
remortel@remortel:~$ ls onbestaand || echo "failure"
ls: cannot access onbestaand: No such file or directory
failure
remortel@remortel:~$ ls bin || echo "failure"
interactive  interactive~  printtest  printtest~  readdtest  readdtest~
remortel@remortel:~$
```

ts reserved.

1.3 Simultaan uitvoeren van programma's

Probleem: Je wil tegelijk 3 onafhankelijke programma's prog1, prog2 en prog3 uitvoeren

Oplossing: Voer ze uit in de achtergrond via & teken

>Prog1 &

>Prog2 &

>Prog 3 &

Probleem: programma's duren zo lang dat ik ze wil laten uitvoeren nadat ik ben uitgelogd op het systeem

Oplossing: vertel de shell om voor je programma het 'hangup' signaal te negeren:

> nohup prog1 &

Be nice to others: rekenintensieve programma's kunnen systemen zwaar belasten → nice -n (n=prioriteit; -20 hoogste, 19 laagste)

> nohup nice -20 prog1 &

© 2008 Pearson Education, Inc. All rights reserved.

1.3 Standaard shell variabelen

- Volledige lijst via commando env
- $PATH: een lijst van locaties waar de shell automatisch in zoekt naar commando's, bibliotheken, manpages beschrijvingen etc.
- $OSTYPE:type van het operating system
- $USER: je gebruikersnaam
- $SHELL: je shell
- $PWD: folder waarin je je nu bevindt
- $OLDPWD: folder waarin je je net hiervoor bevond
- $-: string met alle shell optie vlaggen (bevat letter 'i' indien je interactief met de shell werkt)
- $PS1: vorm en inhoud van de command prompt

…

- $1 - $9 these variables are the positional parameters.
- $0 the name of the command currently being executed.
- $# the number of positional arguments given to this invocation of the shell.
- $? the exit status of the last command executed is given as a decimal string. When a command completes successfully, it returns the exit status of 0 (zero), otherwise it returns a non-zero exit status.
- $$ the process number of this shell - useful for including in filenames, to make them unique.
- $! the process id of the last command run in the background.
- $* a string containing all the arguments to the shell, starting at $1.
- $@ same as above, except when quoted.

© 2008 Pearson Education, Inc. All rights reserved.

1.4 Werken met variabelen

- **Bash kent geen expliciete type declaratie**
- **Alle variabelen zijn strings van karakters**
- **Naargelang de context, zal bash de variabelen behandelen als een bepaald type**
- **Een variabele heeft een *naam* en na initialisatie ook een *waarde***
- ***Indien men de waarde geassocieerd met een naam wil gebruiken voegt men een $ teken in vlak voor de naam:***

  variabele1=1234

  echo variabele1 ***(print het woord variabele1)***

  echo $variabele ***(print 1234)***
- ***Let op! initialisatie gebeurt met het = teken zonder spaties!***

© 2008 Pearson Education, Inc. All rights reserved.

1.4 Bash scripts

- **Bash scrips zijn een verzameling van bash shell commando's, in combinatie met variabelen en controle structuren**
- **Scripts laten toe om een lange set van shell commando's the groeperen en te hergebruiken**
- **Shell scripts worden niet gecompileerd, maar geinterpreteerd door de shell**
- **Basis structuur van een shell script:**
  #!/bin/bash → indicatie dat het een bash script betreft
  …
  # comments → Commentaar voorafgegaan door hekje
  name=value →Initialisatie van een variabele
  …
  <command> $name →gebruik van de waarde van de variabele met commando
  ….
  <control structure> → If … else, for loops, while loops, case switch, etc.

© 2008 Pearson Education, Inc. All rights reserved.

1.4 Eerste bash script

```
#!/bin/bash
#Printing user defined variables

MYTEXT="GigaHerz"
MYFREQ=1.92735
#use export MYFREQ if you want other scripts besides this one to 'knc
#about the value of MYFREQ

printf '%-10.10s = %4.2f\n' $MYTEXT $MYFREQ

#variables can also be combined, use {} for improved readability
MYFILE=${PWD}/bin/variables

echo 'My File is ' $MYFILE

#variables can also be passed to the script as parameters
# use as ./variables <parameter>
# $# is another special vraiable that holds the number of arguments
# passed on to a script

if (( $# == 1))
then
    printf '%s %4.2f\n' "My input was " $1
fi
```

Geen type declaratie
Geen spaties!

Identiek aan C-style
Output format

{ } ter isolatie van
variabele

$# variable bevat aantal
argumenten

Selectie syntax

Fig. 1.2 | Bash script met variabelen en user input.

© 2008 Pearson Education, Inc. All rights reserved.

1.4 Loops over argumenten

File Edit Options Buffers Tools Insert Help

```
#!/bin/bash
#Printing user defined variables


printf '%s %4.2f\n' "My number of inputs is " $#

COUNT=0

for ARG in $*
do
    let COUNT+=1
    printf "Argument %d   is %s\n" $COUNT $ARG
done
```

$* lijst van argumenten ARG is zelfgedefinieerde variabele

Rekenen met integers via let sleutelwoord

For loop syntax

Fig. 1.3 | Bash script met for loop over input argumenten.

© 2008 Pearson Education, Inc. All rights reserved.

1.4 Loops over strings met spaties

```
#!/bin/bash
#Printing a list of arguments (with spaces in names) provided by user
#test by making a directory ~/tmp
#put 4 files in it by
#touch ~/tmp/file1.txt ~/tmp/file2.txt ~/tmp/file3.txt ~/tmp/"File name 1.txt"
#the use teh script by typing
#./loopingnameswithspaces ~/tmp/*.txt


printf '%s %4.2f\n' "My number of inputs is " $#

COUNT=0

for ARG in "$@"
do
    let COUNT+=1
    printf "Argument %d   is %s\n" $COUNT "$ARG"
done
```

"$@" beschouwt namen met spaties als 1 string

Let op aanhalingstekens om namen met spaties als 1 string weer te geven

Fig. 1.4 | Veralgemening van script 1.3

© 2008 Pearson Education, Inc. All rights reserved.

1.4 Optie vlaggen

Stel: Je wil een script dat bij uitvoering ./script <argumenten> geen printouts geeft, maar je wil ook een optievlag -v om wel printouts te geven: ./script -v <argumenten>

```
#!/bin/bash
#Printing a list of arguments (with spaces in names) provided by user
#test by making a directory ~/tmp
#put 4 files in it by
#touch ~/tmp/file1.txt ~/tmp/file2.txt ~/tmp/file3.txt ~/tmp/"File name 1.txt"
#the use teh script by typing
#try ./loopingwithflags -v ~/tmp/*.txt
#  and ./loopingwithflags ~/tmp/*.txt

#parse the option flag
#shift will remove the first option from teh argument list
# so you can loop over the rest like normally

VERBOSE=0
if [[ $1 = -v ]]
then
    VERBOSE=1
    shift
fi

COUNT=0

for ARG in "$@"
do
    let COUNT+=1
    if (( VERBOSE ))
    then
        printf "Argument %d   is %s\n" $COUNT "$ARG"
    fi
done
```

Alternatief voor
If (($1 == -v))
[ ] is equivalent aan commando test
Check man test voor opties

Shift verwijdert $1 uit de argumentenlijst $@

Fig. 1.5 | Veralgemening van script 1.4

© 2008 Pearson Education, Inc. All rights reserved.

1.4 Eenvoudige string manipulatie

Stel: Je wil de namen van een reeks files: filen.txt vervangen in filen.batch

```
#!/bin/bash
# Simple manipulation of file names in a single directory
# To try: mkdir ~/tmp ; touch ~/tmp/file1.txt ~/tmp/file2.txt ~/tmp/file3.txt
# then .simplestringmanip ~/tmp/*.txt

VERBOSE=0
if [[ $1 = -v ]]
then
    VERBOSE=1
    shift
fi

COUNT=0

for ARG in "$@"
do
    let COUNT+=1
    NOTXT=${ARG%txt}
    NEWNAME=${NOTXT}batch
    if (( VERBOSE ))
    then
        printf "Replacing File %s with %s\n" "$ARG" "$NEWNAME"
        mv "$ARG" "$NEWNAME"
    fi
done
if (( VERBOSE ))
    then
        printf "Replaced %d Files\n" $COUNT
fi
```

Trek de substring txt af van het einde van de string in $ARG

Plak de string batch aan de $NOTXT string

Fig. 1.5 | Simpele string manipulatie

© 2008 Pearson Education, Inc. All rights reserved.

1.4 Eenvoudige string manipulatie

Andere varianten:

${#VAR} lengte van de string in VAR

${VAR#<pattern>} verwijder kortste versie van <pattern> string vooraan de string VAR

${VAR##<pattern>} verwijder langste versie van <pattern> string vooraan de string VAR

${VAR%<pattern>} verwijder kortste versie van <pattern> string achteraan de string VAR

${VAR%%<pattern>} verwijder langste versie van <pattern> string achteraan de string VAR

${VAR/<pattern>/string} vervang eerste overeenkomst van <pattern> met string

${VAR//<pattern>/string} vervang alle overeenkomst van <pattern> met string

© 2008 Pearson Education, Inc. All rights reserved.

1.4 Testen van string karakteristieken

```
#!/bin/bash
#
#Testing the properties of strings
#
# zero or nonzero length  (2 equivalent ways)


VAR="$1"
#
if [ "$VAR" ]
then
echo has text
else
echo zero length
fi
#
if [ -z "$VAR" ]
then
echo zero length
else
echo has text
fi
#
#beware when testing equalities, do you want strings to be treated
#as strings or numbers??
#
VAR1=" 05 "
VAR2="5"
printf "%s" "do they -eq as equal? "
if [ "$VAR1" -eq "$VAR2" ]
then
echo YES
else
echo NO
fi
printf "%s" "do they = as equal? "
if [ "$VAR1" == "$VAR2" ]
then
echo YES
else
echo NO
fi

#
# testing for pattern matches
#
MYFILENAME="testfile.jpg"
if [[ "${MYFILENAME}" == *.jpg ]]
then
printf "file %s found \n" "$MYFILENAME"
fi
```

-z levert een 1 op wanneer de lengte van een string nul is

Verschillend gedrag, naargelang gebruik van -eq of ==

Ook wildcards zijn toegestaan

Fig. 1.6 | testen van strings

son Education, Inc. All rights reserved.

1.4 Testen van File karakteristieken

Stel: Je wil alle permissies veranderen van files die write permissie hebben

```
#!/bin/bash
# kijk in directory ~/tmp en ontneem alle files hun write permissie

VERBOSE=0
if [[ $1 = -v ]]
then
    VERBOSE=1
    shift
fi

DIRPLACE="${HOME}"/tmp


if [ -d "$DIRPLACE" ]
then
   cd "$DIRPLACE"
   COUNT=0
   for ARG in "$DIRPLACE"
   do
        [ -w "$ARG"  ] \
  && { printf "Revoking write from %s\n" "$ARG"; \
       let COUNT+=1 ; chmod a-w "$ARG"; }
   done
fi



if (( VERBOSE ))
    then
        printf "Revoked write permission from %d Files\n" $COUNT
fi
```

Check of DIRPLACE een bestaande directory is

Verkorte vorm van if [-w ARG] then …
Check of een file write permissie heeft

Fig. 1.7 | File karakteristieken checken

© 2008 Pearson Education, Inc. All rights reserved.

1.4 Multiple branching

Stel: Je wil een argument testen op een (groot) aantal mogelijkheden

```
#!/bin/bash
#
# Demonstration of branching with case keyword
#
#
MYDIR=$1
for FN in "$MYDIR"
do
    case $FN in
        *.gif) convert $FN ${FN%.gif}.pdf
            ;;
        *.jpg) convert $FN ${FN%.jpg}.pdf
            ;;
        *.tif | *.TIFF) convert $FN ${FN%.tif}.pdf
            ;;
        *) printf "File not supported: %s" $FN
            ;;
    esac
done
```

Case is een C-style variant op de if then selectie

Laatste mogelijkheid *) behandelt alle overblijvende mogelijkheden

Opmerking: case statements kunnen zoals if ook patterns gebruiken met wildcards

Fig. 1.7 | Multiple branching met case

© 2008 Pearson Education, Inc. All rights reserved.

1.4 Eenvoudige menus

Stel: Je wil een gebruiker een menu met selectie mogelijkheden presenteren

```
#!/bin/bash
#
#
# Displays a list of files in current directory and prompt for which
# file to edit
#
# Set the prompt for the select command
PS3="Type a number or 'q' to quit: "

# Create a list of files to display

fileList=$(find . -maxdepth 1 -type f)

# Show a menu and ask for input. If the user entered a valid choice,

# then invoke the editor on that file

select fileName in $fileList;
do

    if [ -n "$fileName" ]; then

        emacs ${fileName}

    fi

    break

done
```

PS3 is de selectie-prompt
Hij wordt hier aangepast met een nuttige instructie voor de gebruiker

Find commando laat complexe zoekopdrachten toe. In dit geval zoeken we naar alle files in de huidige directory

Select commando zal automaisch alle opties uit een lijst strings presenteren
En de gebruiker vragen er 1 te kiezen

Fig. 1.7 | Multiple branching met case

© 2008 Pearson Education, Inc. All rights reserved.

1.4 While Loops

```
#!/bin/bash
#
# Demonstration of different kinds of loops
#
#
COUNT=0
MAX=5
while (( COUNT < MAX ))
do
let COUNT++
printf " Count is %d\n" $COUNT
done


## loop over all lines of a file
#make first a testfile.txt with some lines of text
#
COUNT=0
cat mytestfile.txt | \
while read MYLINE
do
    let COUNT++
    printf "My Line nr %d contains: %s\n" $COUNT "$MYLINE"

done
## is equivalent with, mind that the if statement ignores empt
#COUNT=0
while  read MYLINE
do
    if [ $MYLINE ]
    then
        let COUNT++
        printf "Line nr %d contains: %s\n" $COUNT "$MYLINE"
    fi
done < mytestfile.txt
```

Eenvoudige while loop

Read commando leest elke lijn

While loop is niet meer dan een script op zich
Je kan output van een script via de pipe | doorgeven aan een ander

Je kan ook een file als input geven aan het 'while' script
Check met if [$MYLINE] of de lijn effectief leesbare karakters bevat

Fig. 1.8 | While loops

© 2008 Pearson Education, Inc. All rights reserved.

1.4 For Loops

```
#!/bin/bash
#
# Demonstration of different kinds of loops
#
#
for (( i=0 ; i < 10 ; i++ ))
do
    echo $i
done
## works also with list
for i in a b c d e
do
    echo $i
done
##
# several counters can be incremented simultaneously
# mind the double brackets for evaulating arithmetic expressio
# watch out: this is not identical to two nested for loops
for (( i=0, j=0 ; i+j < 10 ; i++, j++ ))
do
    echo $((i*j))
done
## for floating point arithmetic, use seq to generate the numbers
## counting from 1 to 1.1 in steps of 0.01
## arbitary precision arithmetic can be performed by the bc command
for fp in $(seq 1.0 .01 1.1)
do
    echo $fp
# raise fp to power 2
    echo "scale=20; $fp^2" | bc
done
```

C-style Counter controlled for loop

Wiskundige bewerkingen in een subshell ( )

Seq kan automatisch floating point lijsten maken
Floating point bewerkingen met bc programma
Scale bepaalt de precisie!

Fig. 1.9 | For loops

© 2008 Pearson Education, Inc. All rights reserved.

1.5 Finding files with find

find is een zeer krachtig zoekprogramma om bestanden terug te vinden op je systeem

Syntax: *find [path ..] [expression]*

[path]: het startpunt vanwaar find begint te zoeken (bvb / voor het hele systeem, of $HOME voor je home directory), jan kan meerdere paths geven, gescheiden door spaties

[expression]: bestaat uit options die geldig zijn voor de hele opdracht, tests die voor elke file een true of false geven, en actions.

Deze zijn door logische operatoren (-and, -or, -not gescheiden, indien geen gegeven wordt –and verondersteld)

Meer info via *man find*

Negeren van waarschuwingen en fouten: Indien find een file niet kan inspecteren (wegens gebrek aan permissie bvb) genereert het een foutenboodschap. Deze kan je naar /dev/null sturen via

find [path ..] [expression] 2>/dev/null

© 2008 Pearson Education, Inc. All rights reserved.

1.5 Finding files with find

Voorbeelden van options:

-maxdepth n: zoek maximaal n levels diep in het path

-mindepth n: zoek minimaal n levels diep in het path

Voorbeelden van tests:

-amin n: file is n minuten geleden laatst gemanipuleerd

-atime n: file is n*24 uur laatst gemanipuleerd

-name [expr]: file heeft naam [expr] (kan wildcards bevatten)

-iname [expr]: zelfde als name maar case insensitive

-user [uname]: file hoort aan user [uname] toe

Voorbeelden van actions:

-delete: verwijderd de files waarvoor voorgaande tests true geven

-print: print de volledige naam (inclusief path)van de file op het scherm

-exec <command> {}: voer een commando uit op de files, bvb convert ….

© 2008 Pearson Education, Inc. All rights reserved.

1.6 Practicum

Download de file voorbeelden.tar uit Les1 op Blackboard naar je Linux systeem, zet deze in een folder die je zelf maakt en pak uit met

tar –xvf voorbeelden.tar

1. Kijk eens wat tar precies doet met *man tar*
2. Open de files met de emacs editor via: emacs <filenaam> &
3. Voer elk script eens uit via command line ./<scriptnaam>

© 2008 Pearson Education, Inc. All rights reserved.

1.6 Practicum

Bekijk vervolgens de volgende voorbeelden, maak veranderingen in de broncode aan en observeer het gevolg:

1. Printtest: speel met de printf format opties
2. Variables: speel met argumenten aan je script
3. Loopingvariables: laat ‘let’ weg in counting en test opnieuw
4. Loopingnameswithspaces: vervang “$@” door $*
5. Loopingwithflags: laat ‘shift’ instructie weg
6. Simplestringmanip: vervang “$DIRPLACE” door “$DIRPLACE”/*
7. Stringtesting: observeer goed verschil tussen == en –eq
8. Simplemenu: bekijk de opties van commando find met *man find*
9. Looping: vervang $MYLINE door “$MYLINE” in if statement
10. Looping2: bekijk het gebruik van bc programma via google

© 2008 Pearson Education, Inc. All rights reserved.

Te Onthouden

1. Linux file systeem
2. Basiscommando's in Linux
3. Gebruik van man pages
4. De bash shell en haar syntax

© 2008 Pearson Education, Inc. All rights reserved.